



SQL is case-insensitive and space-insensitive: you can write it in all uppercase or all lowercase, with however many horizontal and vertical spaces you want. In this book, SQL keywords will be written in uppercase and the statements will be formatted to read cleanly.

A glance at the psql commands

Every command specific to psql starts with a backslash character (\). It is possible to get some help with SQL statements and PostgreSQL commands via the special \h command, after which you can specify the specific statement you want help for:

```
template1=> \h SELECT
Command:      SELECT
Description: retrieve rows from a table or view
Syntax:
[ WITH [ RECURSIVE ] with_query [, ...] ]
SELECT [ ALL | DISTINCT [ ON ( expression [, ...] ) ] ]
        [ * | expression [ [ AS ] output_name ] [, ...] ]
...
URL: https://www.postgresql.org/docs/16/sql-select.html
```



The displayed help is, for space reasons, concise. You can find a much more verbose description and usage examples in the online documentation. For this reason, at the end of the help screen, there is a link reference to the online documentation.

If you need help with the psql commands, you can issue a \? command:

```
template1=> \?
General
  \copyright          show PostgreSQL usage and distribution terms
  \crosstabview [COLUMNS] execute query and display results in crosstab
  \errverbose         show most recent error message at maximum
verbosity
  \g [FILE] or ;      execute query (and send results to file or |pipe)
  \gdesc              describe result of query, without executing it
...
```

There are also a lot of *introspection* commands, such as, for example, `\d` to list all user-defined tables. These special commands are, under the hood, a way to execute queries against the PostgreSQL system catalogs, which are, in turn, registries about all objects that live in a database. The introspection commands will be shown later in the book and are useful as shortcuts to get an idea of which objects are defined in the current database.

Many `psql` features will be detailed as you move on through the book, but it is worth spending some time trying to get used to this very efficient and rich command-line client.

Introducing the connection string

In the previous section, you learned how to specify basic connection options, such as `-d` and `-U` for a database and user, respectively. `psql` also accepts a LibPQ connection string.

LibPQ is the underlying library that every application can use to connect to a PostgreSQL cluster and is, for example, used in C and C++ clients, as well as non-native connectors.

A connection string in LibPQ is a URI made up of several parts:

```
postgresql://username@host:port/database
```

Here, we have the following:

- `postgresql` is a fixed string that specifies the protocol the URI refers to.
- `username` is the PostgreSQL username to use when connecting to the database.
- `host` is the hostname (or IP address) to connect to.
- `port` is the TCP/IP port the server is listening on (by default, 5432).
- `database` is the name of the database to which you want to connect.

The username, port, and database parts can be omitted if they are set to their default (the username is the same as the operating system username).

The following connections are all equivalent:

```
$ psql -d template1 -U luca -h localhost
$ psql postgresql://luca@localhost/template1
$ psql postgresql://luca@localhost:5432/template1
```

Solving common connection problems

There are a few common problems when dealing with database connections, and this section explains them in order to ease your task of getting connected to your cluster.

Please note that the solutions provided here are just for testing purposes and not for production usage. All of the security settings will be explained in later chapters, so the aim of the following subsection is just to help you get your test environment usable.

Database “foo” does not exist

This means either you misspelled the name of the database in the connection string or you are trying to connect without specifying the database name.

For instance, the following connection fails when executed by an operating system user named luca because, by default, it is assuming that the user luca is trying to connect to a database with the same name (meaning luca) since none has been explicitly set:

```
$ psql
psql: error: could not connect to server: FATAL: database "luca" does not exist
```

The solution is to provide an existing database name via the `-d` option or to create a database with the same name as the user.

Connection refused

This usually means there is a network connection problem, so either the host you are trying to connect to is not reachable or the cluster is not listening on the network.

As an example, imagine PostgreSQL is running on a machine named venkman and we are trying to connect from another host on the same network:

```
$ psql -h venkman -U luca template1
psql: error: could not connect to server: could not connect to server:
Connection refused

        Is the server running on host "venkman" (192.168.222.123) and
        accepting
        TCP/IP connections on port 5432?
```

In this case, the database cluster is running on the remote host but is not accepting connections from the outside. Usually, you have to fix the server configuration or connect to the remote machine (via SSH, for instance) and open a local connection from there.

In order to quickly solve the problem, you have to edit the `postgresql.conf` file (usually located under the `PGDATA` directory) and ensure the `listen_address` option has an asterisk (or the name of your external network card) so that the server will listen on any available network address:

```
listen_addresses = '*'
```

After a restart of the service, by means of the `restart` command issued to `pg_ctl`, the client will be able to connect. Please note that enabling the server to listen on any available network address might not be the optimal solution and can expose the server to risks in a production environment. Later in the book, you will learn how to specifically configure the connection properties for your server.

No `pg_hba.conf` entry

This error means the server is up and running and able to accept your request, but the PostgreSQL built-in **Host-Based Access (HBA)** control does not permit you to enter.



This error should never happen in the Docker container used for this chapter, because its configuration is already allowing trusted connections. However, other PostgreSQL installations will be stricter; therefore, knowing about this type of error message can help you to quickly figure out where the configuration problem is.

As an example, the following connection is refused:

```
$ psql -h localhost -U luca template1
psql: error: could not connect to server: FATAL: no pg_hba.conf entry for
host "127.0.0.1", user "luca", database "template1", SSL off
```

The reason for this is that, inspecting the `pg_hba.conf` file, there is no rule to let the user `luca` in on the `localhost` interface. So, for instance, adding a single line such as the following to the `pg_hba.conf` file can fix the problem:

```
host all luca 127.0.0.1/32 trust
```

You need to reload the configuration in order to apply changes. The format of every line in the `pg_hba.conf` file will be discussed later, but for now, please assume that the preceding line instructs the cluster to accept any connection incoming from `localhost` by means of the user `luca`.

Exploring the disk layout of PGDATA

In the previous sections, you have seen how to install PostgreSQL and connect to it, but we have not looked at the storage part of a cluster. Since the aim of PostgreSQL, as well as the aim of any relational database, is to permanently store data, the cluster needs some sort of permanent storage. In particular, PostgreSQL exploits the underlying filesystem to store its own data. All of the PostgreSQL-related stuff is contained in a directory known as PGDATA.

The PGDATA directory acts as the disk container that stores all the data of the cluster, including the users' data and cluster configuration.

The following is an example of the content of PGDATA for a running PostgreSQL 16 cluster:

```
$ ls -1 /postgres/16/data
base
global
pg_commit_ts
pg_dynshmem
pg_hba.conf
pg_ident.conf
pg_logical
pg_multixact
pg_notify
pg_replslot
pg_serial
pg_snapshots
pg_stat
pg_stat_tmp
pg_subtrans
pg_tblspc
pg_twophase
PG_VERSION
pg_wal
pg_xact
postgresql.auto.conf
postgresql.conf
postmaster.opts
postmaster.pid
```

The PGDATA directory is structured in several files and subdirectories. The main files are as follows:

- `postgresql.conf` is the main configuration file, used by default when the service is started.
- `postgresql.auto.conf` is the automatically included configuration file used to store dynamically changed settings via SQL instructions.
- `pg_hba.conf` is the HBA file that provides the configuration regarding available database connections.
- `PG_VERSION` is a text file that contains the major version number (useful when inspecting the directory to understand which version of the cluster has managed the PGDATA directory).
- `postmaster.pid` is the PID of the postmaster process, the first launched process in the cluster.

The main directories available in PGDATA are as follows:

- `base` is a directory that contains all the users' data, including databases, tables, and other objects.
- `global` is a directory containing cluster-wide objects.
- `pg_wal` is the directory containing the WAL files.
- `pg_stat` and `pg_stat_tmp` are, respectively, the storage of permanent and temporary statistical information about the status and health of the cluster.

Of course, all files and directories in PGDATA are important for the cluster to work properly, but so far, the preceding is the “core” list of objects that are fundamental in PGDATA itself. Other files and directories will be discussed in later chapters.

Objects in the PGDATA directory

PostgreSQL does not name objects on disk, such as tables, in a mnemonic or human-readable way; instead, every file is named after a numeric identifier. You can see this by having a look, for instance, at the `base` subdirectory:

```
$ ls -l /postgres/16/data/base
1
16386
4
5
```

As you can see from the preceding code, the base directory contains four objects, named 1,4, 5, and 16386. Please note that these numbers could be different on your machine. In particular, each of the preceding is a directory that contains other files, as shown here:

```
$ ls -l /postgres/16/data/base/16386 | head
112
113
1247
1247_fsm
1247_vm
1249
1249_fsm
1249_vm
1255
1255_fsm
```

As you can see, each file is named with a numeric identifier. Internally, PostgreSQL holds a specific catalog that allows the database to match a mnemonic name to a numeric identifier, and vice versa. The integer identifier is named **OID** (or, **Object Identifier**); this name is a historical term that today corresponds to the so-called *filenode*. The two terms will be used interchangeably in this section.

There is a specific utility that allows you to inspect a PGDATA directory and extract mnemonic names: `oid2name`. For example, if you executed the `oid2name` utility, you'd get a list of all available databases similar to the following one:

```
$ oid2name
All databases:
  Oid  Database Name  Tablespace
-----
16390      forumdb  pg_default
5         postgres  pg_default
4      template0  pg_default
1      template1  pg_default
```

As you can see, the `Oid` numbers in the `oid2name` output reflect the same directory names listed in the base directory; every subdirectory has a name corresponding to the database.

You can even go further and inspect a single file going into the database directory, specifying the database where you are going to search for an object name with the `-d` flag:

```
$ cd /postgres/16/data/base/1
$ oid2name -d template1 -f 3395
From database "template1":
  Filenode              Table Name
-----
  3395  pg_init_privs_o_c_o_index
```

As you can see from the preceding example, the 3395 file in the `/postgres/16/data/base/1` directory corresponds to the table named `pg_init_privs_o_c_o_index`. Therefore, when PostgreSQL needs to interact with a table like this, it will seek the disk to the `/postgres/16/data/base/1/3395` file.

From the preceding example, it should be clear that every SQL table is stored as a file with a numeric name. However, PostgreSQL does not allow a single file to be greater than 1 GB in size, so what happens if a table grows beyond that limit? PostgreSQL “attaches” another file with a numeric extension that indicates the next chunk of 1 GB of data. In other words, if your table is stored in the 123 file, the second gigabyte will be stored in the 123.1 file, and if another gigabyte of storage is needed, another file, 123.2, will be created. Therefore, the filenode refers to the very first file related to a specific table, but more than one file can be stored on disk.

Tablespaces

PostgreSQL pretends to find all its data within the PGDATA directory, but that does not mean that your cluster is “jailed” in this directory. In fact, PostgreSQL allows “escaping” the PGDATA directory by means of **tablespaces**. A tablespace is a directory that can be outside the PGDATA directory and can also belong to different storage. Tablespaces are mapped into the PGDATA directory by means of symbolic links stored in the `pg_tblspc` subdirectory. In this way, the PostgreSQL processes do not have to look outside PGDATA, but are still able to access “external” storage. A tablespace can be used to achieve different aims, such as enlarging the storage data or providing different storage performances for specific objects. For instance, you can create a tablespace on a slow disk to contain infrequently accessed objects and tables, keeping fast storage within another tablespace for frequently accessed objects.

You don’t have to make links by yourself: PostgreSQL provides the `TABLESPACE` feature to manage this and the cluster will create and manage the appropriate links under the `pg_tblspc` subdirectory.

For instance, the following is a PGDATA directory that has three different tablespaces:

```
$ ls -l /postgres/16/data/pg_tblspc/
lrwxrwxrwx 1 postgres postgres 22 Jan 19 13:08 16384 -> /data/tablespaces/
ts_a
lrwxrwxrwx 1 postgres postgres 22 Jan 19 13:08 16385 -> /data/tablespaces/
ts_b
lrwxrwxrwx 1 postgres postgres 22 Jan 19 13:08 16386 -> /data/tablespaces/
ts_c
```

As you can see from the preceding example, there are three tablespaces that are attached to the /data storage. You can inspect them with `oid2name` and the `-s` flag:

```
$ oid2name -s
All tablespaces:
  Oid  Tablespace Name
-----
 1663      pg_default
 1664      pg_global
16384          ts_a
16385          ts_b
16386          ts_c
```

As you can see, the numeric identifiers of the symbolic links are mapped to the mnemonic names of the tablespaces. From the preceding example, you can observe that there are also two particular tablespaces:

- `pg_default` is the default tablespace corresponding to “none,” the default storage to be used for every object when nothing is explicitly specified. In other words, every object stored directly under the PGDATA directory is attached to the `pg_default` tablespace.
- `pg_global` is the tablespace used for system-wide objects.

By default, both of the preceding tablespaces refer directly to the PGDATA directory, meaning any cluster without a custom tablespace is totally contained within the PGDATA directory.

Exploring configuration files and parameters

The main configuration file for PostgreSQL is `postgresql.conf`, a text-based file that drives the cluster when it starts.

Usually, when changing the configuration of the cluster, you must edit the `postgresql.conf` file to write the new settings and, depending on the context of the settings you have edited, to issue a cluster SIGHUP signal (that is, *reload* the configuration) or restart it.

Every configuration parameter is associated with a *context*, and depending on the context, you can apply changes with or without a cluster restart. Available contexts are as follows:

- **internal:** A group of parameters that are set at compile time and therefore cannot be changed at runtime.
- **postmaster:** All the parameters that require the cluster to be restarted (that is, to kill the postmaster process and start it again) to activate them.
- **sighup:** All the configuration parameters that can be applied with a SIGHUP signal sent to the postmaster process, which is equivalent to issuing a *reload* signal in the operating system service manager.
- **backend and superuser-backend:** All the parameters that can be set at runtime but will be applied to the next normal or administrative connection.
- **user and superuser:** A group of settings that can be changed at runtime and are immediately active for normal and administrative connection.

The configuration parameters will be explained later in the book, but the following is an example of a minimal configuration file with some different settings:

```
$ cat /postgres/16/data/postgresql.conf
shared_buffers = 512MB
maintenance_work_mem = 128MB
checkpoint_completion_target = 0.7
wal_buffers = 16MB
work_mem = 32MB
min_wal_size = 1GB
max_wal_size = 2GB
```

The `postgresql.auto.conf` file has the very same syntax as the main `postgresql.conf` file but is automatically overwritten by PostgreSQL when the configuration is changed at runtime directly within the system, by means of specific administrative statements such as `ALTER SYSTEM`. The `postgresql.auto.conf` file is always loaded at the very last moment, therefore overwriting other settings. In a fresh installation, this file is empty, meaning it will not overwrite any other custom setting.